


# 3 Fundamental Data Structures

*Efficient Algorithms · Wintersemester 2026–27*

Prof. Dr. Sebastian Wild

CS566 (Wintersemester 2026–27)
Philipps-Universität Marburg
*version* 2026-10-27 01:48 H

# Learning Outcomes

**Unit 3:** ***Fundamental Data Structures***

1. Understand and demonstrate the difference between *abstract data type (ADT)* and its *implementation*
2. Be able to define the ADTs *stack*, *queue*, *priority queue* and *dictionary/symbol table*
3. Understand *array*-based implementations of stack and queue
4. Understand *linked lists* and the corresponding implementations of stack and queue
5. Know *binary heaps* and their performance characteristics
6. Understand *binary search trees* and their performance characteristics
7. Know high-level idea of basic *hashing strategies* and their performance characteristics

# Outline

## 3 Fundamental Data Structures


3.1 Stacks & Queues
3.2 Resizable Arrays
3.3 Priority Queues & Binary Heaps
3.4 Operations on Binary Heaps
3.5 Symbol Tables
3.6 Binary Search Trees
3.7 Ordered Symbol Tables
3.8 Balanced BSTs
3.9 Hashing – Basics
3.10 Universal Hashing

# Recap: The Random Access Machine

- Data structures make heavy use of pointers and dynamically allocated memory.

- Recall: Our RAM model supports
  - basic pseudocode (≈ simple Python/Java code)
  - creating arrays of a fixed/known size.
  - creating instances (objects) of a known class.



Python abstracts this away!

no predefined capacity!

**There are *no arrays* in Python, only its built-in *lists*.**

But: Python *implementations create* lists based on fixed-size arrays (stay tuned!)



Python ≠ RAM: Not every built-in Python instruction runs in $O(1)$ time!

# 3.1 Stacks & Queues

# Abstract Data Types

**abstract data type (ADT)**

- list of supported operations
- **what** should happen
- **not:** how to do it
- **not:** how to store data

≈ Java `interface`, Python `ABC`s (with comments)

abstract base classes

VS.

**data structures**

- specify exactly **how** data is represented
- **algorithms** for operations
- has concrete costs (space and running time)

≈ Java/Python `class` (non abstract)

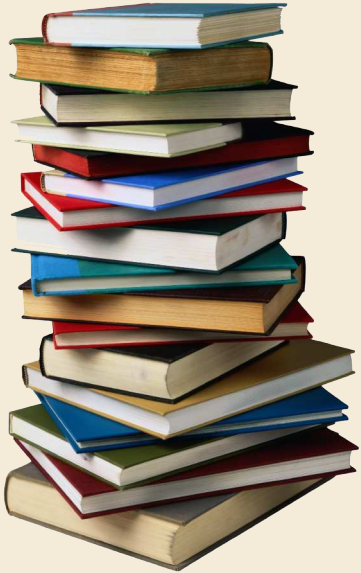
*Why separate?*

- Can swap out implementations ⇝ "drop-in replacements"

⇝ **reusable code**!

- (Often) better abstractions
- Prove generic lower bounds ( ⇝ Unit 4)

# Stacks



**Stack ADT**

- `top()`
  Return the topmost item on the stack
  Does not modify the stack.
- `push(`$x$`)`
  Add $x$ onto the top of the stack.
- `pop()`
  Remove the topmost item from the stack
  (and return it).
- `isEmpty()`
  Returns `true` iff stack is empty.
- `create()`
  Create and return an new empty stack.

# Linked-list implementation for Stack

**Invariants:**

- maintain pointer *top* to topmost element
- each element points to the element below it (or `null` if bottommost)

```
1  class Node
2      value
3      next
4
5  class Stack
6      top := null
7      procedure top():
8          return top.value
9      procedure push(x):
10         top := new Node(x, top)
11     procedure pop():
12         t := top()
13         top := top.next
14         return t
```

# Linked-list implementation for Stack – Discussion

**Linked stacks:**


require $\Theta(n)$ space when $n$ elements on stack


All operations take $O(1)$ time


require $\Theta(n)$ space when $n$ elements on stack

Can we avoid extra space for pointers?

# Array-based implementation for Stack

If we want no pointers $\rightsquigarrow$ array-based implementation

**Invariants:**

- maintain array $S$ of elements, from bottommost to topmost
- maintain index *top* of position of topmost element in `S`.



What to do if stack is full upon `push`?

**Array stacks:**

- require *fixed capacity* $C$ (decided at creation time)!
- require $\Theta(C)$ space for a capacity of $C$ elements
- all operations take $O(1)$ time

# Queues

**Operations:**

- `enqueue(`$x$`)`
  Add $x$ at the end of the queue.
- `dequeue()`
  Remove item at the front of the queue and return it.



Implementations similar to stacks.

# Bags

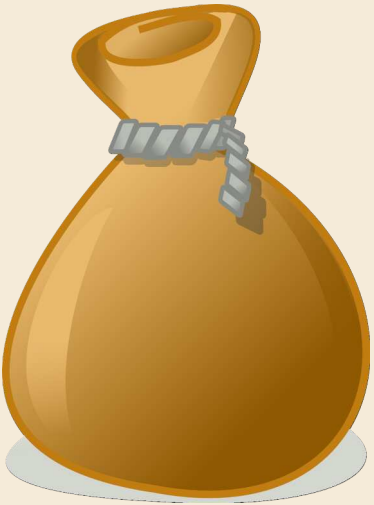
*What do Stack and Queue have in common?*

They are special cases of a ***Bag***!

**Update Operations:**

- `insert(`$x$`)`
  Add $x$ to the items in the bag.
- `delAny()`
  Remove any one item from the bag and return it.
  (Not specified which; any choice is fine.)

- roughly similar to Java's `java.util.Collection`
  Python's `collections.abc.Collection`
- always support iterating over content (read only)



Sometimes it is useful to *state* that order is irrelevant $\rightsquigarrow$ Bag
Implementation of Bag usually just a Stack or a Queue

# 3.2 Resizable Arrays

# *Digression* – Arrays as ADT

Arrays can also be seen as an ADT! . . . but are commonly seen as specific data structure

**Array operations:**

- `create(`$n$`)` *Java*: `A = new int[`$n$`];` *Python:* `A = [0] *` $n$
  Create a new array with $n$ cells, with positions $0, 1, \ldots, n-1$;
  we write $A[0..n) = A[0..n-1]$
- `get(`$i$`)` *Java/Python*: `A[`$i$`]`
  Return the content of cell $i$
- `set(`$i$`,`$x$`)` *Java/Python*: `A[`$i$`] =` $x$`;`
  Set the content of cell $i$ to $x$.

⇝ Arrays have *fixed* size (supplied at creation). (≠ `lists` in Python)

Usually directly implemented by compiler + operating system / virtual machine.



**Difference to "real" ADTs:** ***Implementation usually fixed***
to "a contiguous chunk of memory".

# Doubling trick

*Can we have unbounded stacks based on arrays?* Yes!

**Invariants:**

- maintain array $S$ of elements, from bottommost to topmost
- maintain index *top* of position of topmost element in `S`
- maintain capacity $C = S.length$ so that $\frac{1}{4}C \leq n \leq C$

$\rightsquigarrow$ can always push more elements!

*How to maintain the last invariant?*

- before `push`
  If $n = C$, allocate new array of size $2n$, copy all elements.
- after `pop`
  If $n < \frac{1}{4}C$, allocate new array of size $2n$, copy all elements.

$\rightsquigarrow$ ***"Resizing Arrays"***

an implementation technique, not an ADT!

# Amortized Analysis

- Any individual operation `push` / `pop` can be expensive!
  $\Theta(n)$ time to copy all elements to new array.
- **But:** One expensive operation of cost $T$ means $\Omega(T)$ next operations are cheap!




distance to boundary

since $n \le C \le 4n$

**Formally:** consider "credits/potential" $\Phi = \min\{n - \frac{1}{4}C, C - n\} \in [0, 0.6n]$

- amortized cost of an operation = actual cost (array accesses) $- 4 \cdot$ change in $\Phi$
  - cheap `push`/`pop`: actual cost $1$ array access, consumes $\le 1$ credits $\rightsquigarrow$ amortized cost $\le 5$
  - copying `push`: actual cost $2n+1$ array accesses, creates $\frac{1}{2}n+1$ credits $\rightsquigarrow$ amortized cost $\le 5$
  - copying `pop`: actual cost $2n+1$ array accesses, creates $\frac{1}{2}n-1$ credits $\rightsquigarrow$ amortized cost $5$

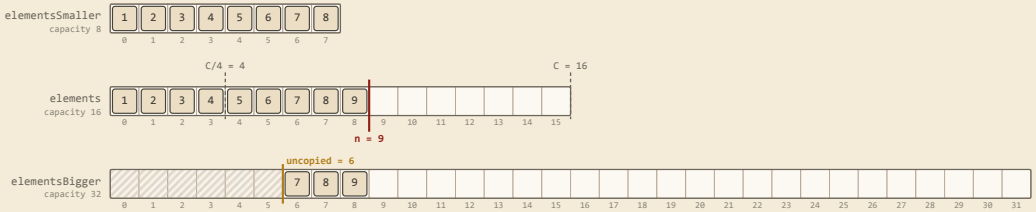
$\rightsquigarrow$ **sequence** of $m$ operations: total actual cost $\le$ total amortized cost + final credits

here: $\le \quad 5m \quad + \quad 4 \cdot 0.6n \quad = \Theta(m+n)$

# Deamortized Resizable Arrays

*What if we need $O(1)$ **worst case** time?*

- It's possible to *de-amortize* the resizing arrays solution!
- maintain **3 arrays**: $S$ (as before) and $S_2$ and $S_{1/2}$ of twice and half the size of $S$




- write operations go to all 3 arrays
- upon resize, "shift" arrays up/down $\rightsquigarrow$ $S_2$ resp. $S_{1/2}$ become new $S$
  - allocate new array, but **delay filling it with elements** ← general strategy!
  - every insert or delete *copies 2 slots* from last resize

$\rightsquigarrow$ by time for next resize, we have caught up and $S_2$ resp. $S_{1/2}$ ready to use

# Deamortized Resizable Arrays – Analysis

**Analysis:**

assuming memory allocation in $O(1)$ $\rightsquigarrow$ needs to be uninitialized!

- $O(1)$ worst case time for read/write by index, push, and pop!
- up to 7 array accesses per operation
- up to $14n$ space

other time-space trade-offs possible

⚠ we assume here that array allocation is $O(1)$ time (no initialization!)

# *Rabbit Hole:* Can we do this more space-efficiently?

- It might appear as if every efficient implementation of a stack needs $\Omega(n)$ extra space on top of space for storing the $n$ elements in the stack.
- But this is not true!
- Can get operations in $O(1)$ worst-case time with $O(\sqrt{n})$ extra space at any time (!)
  - Maintain a collection of small arrays (plus header with pointers to them)
  - Clever choice of block sizes guarantees
    $O(\sqrt{n})$ blocks of $O(\sqrt{n})$ elements throughout
    and fast calculation of address for an index.

    imaginary "superblocks" of sizes $2^k$, $k = 0, 1, \ldots, \lg n$
    $k$th superblock consists of $2^{k/2}$ actual blocks of $2^{k/2}$ elements each.
  - $O(\sqrt{n})$ extra space is best possible



**Brodnik, Carlsson, Demaine, Munro, Sedgewick**: *Resizable Arrays in Optimal Time and Space,* WADS **1999**

# 3.3 Priority Queues & Binary Heaps

# Priority Queue ADT – min-oriented version

Now: elements in the bag have different *priorities*.

**(Max-oriented) Priority Queue (MaxPQ):**

- `construct(`$A$`)`
  Construct from from elements in array $A$.
- `insert(`$x$`,`$p$`)`
  Insert item $x$ with priority $p$ into PQ.
- `max()`
  Return item with largest priority. (Does not modify the PQ.)
- `delMax()`
  Remove the item with largest priority and return it.
- `changeKey(`$x$`,`$p'$`)`
  Update $x$'s priority to $p'$.
  Sometimes restricted to *increasing* priority.
- `isEmpty()`

Fundamental building block in many applications.



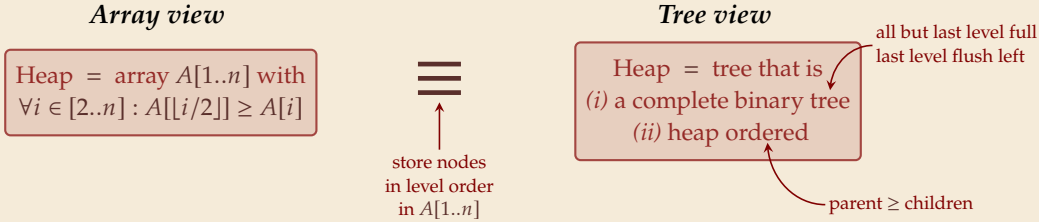
# PQ implementations

**Elementary implementations**

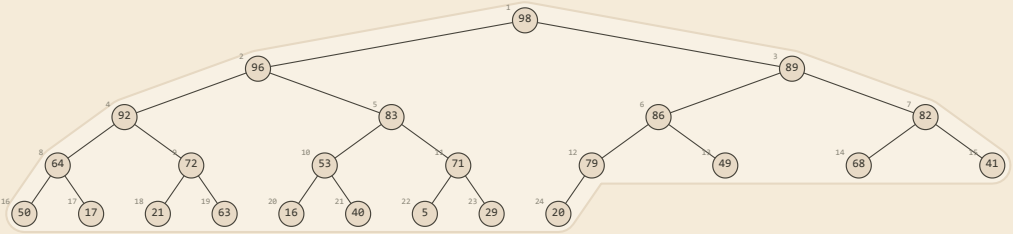
- unordered list $\rightsquigarrow$ $\Theta(1)$ `insert`, but $\Theta(n)$ `delMax`
- sorted list $\rightsquigarrow$ $\Theta(1)$ `delMax`, but $\Theta(n)$ `insert`

*Can we get something between these extremes? Like a "slightly sorted" list?*

**Yes!** *Binary heaps.*



# Binary heap example






# Why heap-shaped trees?

**Why complete binary tree shape?**

- only one possible tree shape $\rightsquigarrow$ keep it simple!
- complete binary trees have minimal height among all binary trees
- simple formulas for moving from a node to parent or children:
  For a node at index $k$ in $A$ ⚠ Recall: nodes at indices $[1..n]$
  - parent at $\lfloor k/2 \rfloor$ (for $k \geq 2$)
  - left child at $2k$
  - right child at $2k + 1$

**Why heap ordered?**

- Maximum must be at root! $\rightsquigarrow$ `max()` is trivial!
- But: Sorted only along paths of the tree; leaves lots of leeway for fast inserts

how? . . . stay tuned

## 3.4 Operations on Binary Heaps

# Insert

1. Add new element at only possible place: bottom-most level, next free spot.
2. Let element *swim* up to repair heap order.

# Delete Max

1. Remove max (must be in root).
2. Move last element (bottom-most, rightmost) into root.
3. Let root key *sink* in heap to repair heap order.

# Heap construction

- $n$ times `insert` $\rightsquigarrow$ *works, but we can do better!*
- instead:
  1. Start with singleton heaps (one element)
  2. Repeatedly merge two heaps of height $k$ with new element into heap of height $k+1$
     (= let the new element *sink*, exactly as in `delMax`)

  $\rightsquigarrow$ in the array: `sink(`$i$`)` for $i = \lfloor n/2 \rfloor, \ldots, 1$ (leaves need no sink)

# Analysis

**Height of binary heaps:**

- *height* of a tree: # edges on longest root-to-leaf path
- *depth/level* of a node: # edges from root $\rightsquigarrow$ root has depth $0$
- How many nodes on first $k$ *full* levels? $\sum_{\ell=0}^{k} 2^\ell = 2^{k+1} - 1$

$\rightsquigarrow$ Height of binary heap: $h \;=\; \min k \text{ s.t. } 2^{k+1} - 1 \geq n \;=\; \lfloor \lg(n) \rfloor$

**Analysis:**

- `insert`: new element "swims" up $\rightsquigarrow$ $\leq h$ steps ($h$ cmps)
- `delMax`: last element "sinks" down $\rightsquigarrow$ $\leq h$ steps ($2h$ cmps)
- `construct` from $n$ elements:

  cost = cost of letting *each node* in heap sink!

$$\leq 1 \cdot h + 2 \cdot (h-1) + 4 \cdot (h-2) + \cdots + 2^\ell \cdot (h-\ell) + \cdots + 2^{h-1} \cdot 1 + 2^h \cdot 0$$

$$= \sum_{\ell=0}^{h} 2^\ell (h-\ell) \;=\; \sum_{i=0}^{h} \frac{2^h}{2^i} i \;=\; 2^h \sum_{i=0}^{h} \frac{i}{2^i} \;\leq\; 2 \cdot 2^h \;\leq\; 2n \text{ steps } (\leq 4n \text{ comparisons})$$

# Binary heap summary

| Operation | Running Time |
|---|---|
| `construct(`$A[1..n]$`)` | $O(n)$ |
| `max()` | $O(1)$ |
| `insert(`$x$`,`$p$`)` | $O(\log n)$ |
| `delMax()` | $O(\log n)$ |
| `changeKey(`$x$`,`$p'$`)` | $O(\log n)$ |
| `isEmpty()` | $O(1)$ |
| `size()` | $O(1)$ |

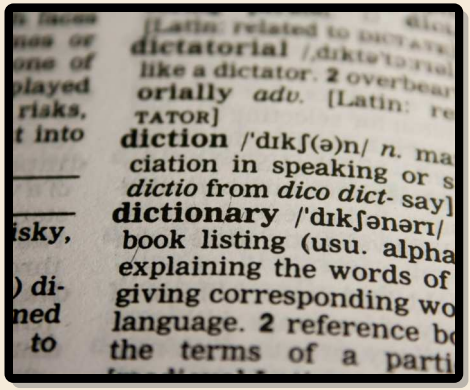
# 3.5 Symbol Tables

# Symbol table ADT

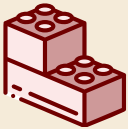
Java: `java.util.Map<K,V>`

**Symbol table / Dictionary / Map / Associative array / key-value store:**

Python dict `{k:v}`




- `put(`$k$`,`$v$`)` *Python* dict: `d[`$k$`] = `$v$
  Put key-value pair $(k, v)$ into table
- `get(`$k$`)` *Python* dict: `d[`$k$`]`
  Return value associated with key $k$
- `delete(`$k$`)` *Python* dict: `del d[`$k$`]`
  Remove key $k$ (any associated value) form table
- `contains(`$k$`)` *Python* dict: $k$ `in d`
  Returns whether the table has a value for key $k$
- `isEmpty(),size()`
- `create()`



*Most fundamental building block in computer science.*

(Every programming library has a symbol table implementation.)

# Symbol tables vs. mathematical functions

- similar interface
- but: mathematical functions are *static/immutable* (never change their mapping)
  (Different mapping is a *different* function)
- symbol table = *dynamic* mapping
  Function may change over time

# Elementary implementations

**Unordered (linked) list:**

- 👍 Fast `put`
- 👎 $\Theta(n)$ time for `get`
- $\rightsquigarrow$ Too slow to be useful

**Sorted *linked* list:**

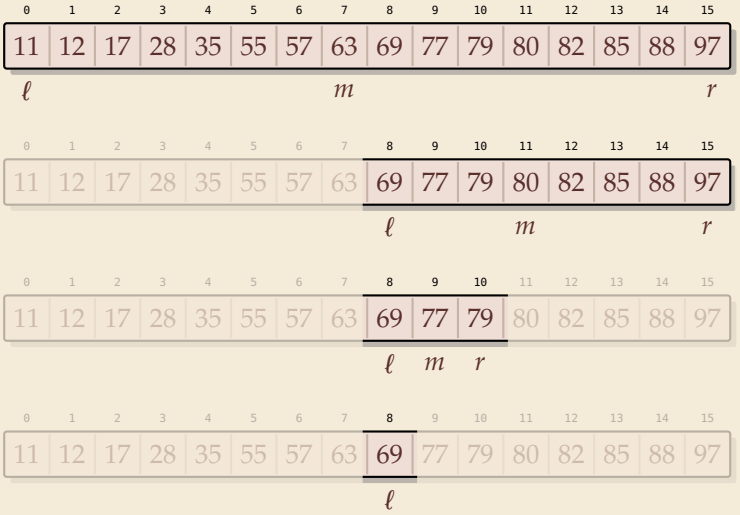
- 👎 $\Theta(n)$ time for `put`
- 👎 $\Theta(n)$ time for `get`
- $\rightsquigarrow$ Too slow to be useful

- $\rightsquigarrow$ *Sorted order does not help us at all?!*

# Binary search

*It does help . . . if we have a sorted **array**!*

**Example:** search for 69




**Binary search**:

- halve remaining list in each step ($\pm 1$)

$\rightsquigarrow \leq \lfloor \lg n \rfloor + 1$ cmps in the worst case



needs random access!

# 3.6 Binary Search Trees

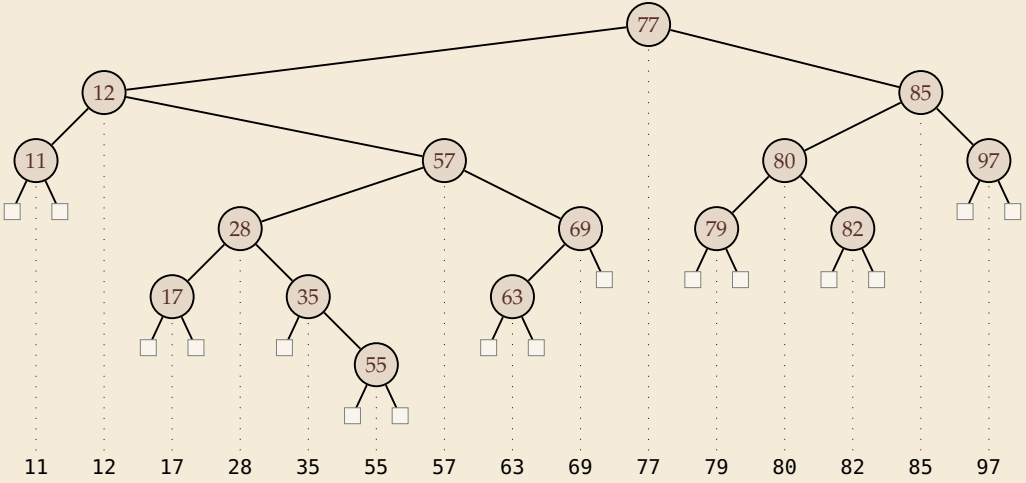
# Binary search trees

**Binary search trees (BSTs)** $\approx$ dynamic sorted array

- binary tree
  - Each node has left and right child
  - Either can be empty (`null`)
- Keys satisfy ***search-tree property***

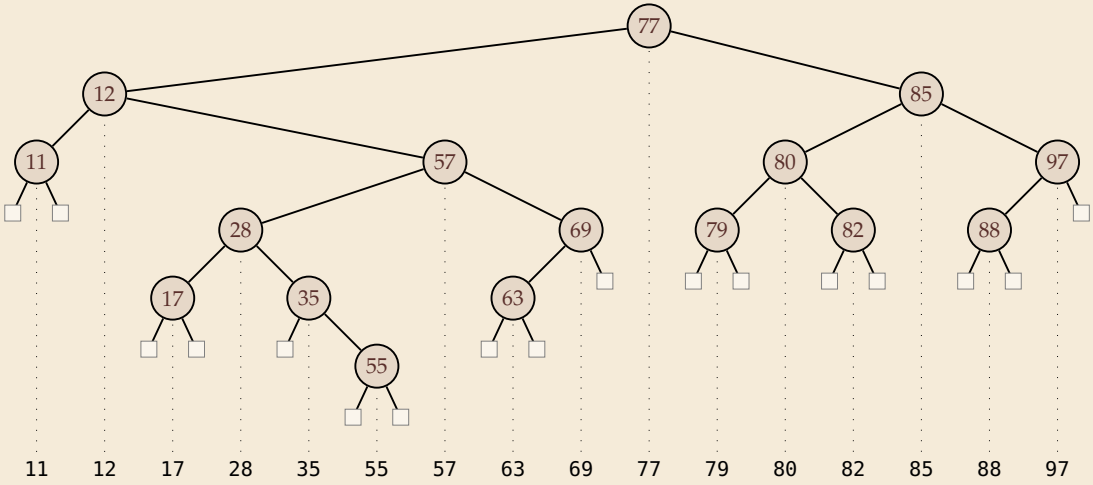
  all keys in left subtree $\leq$ root key $\leq$ all keys in right subtree

# BST example & find



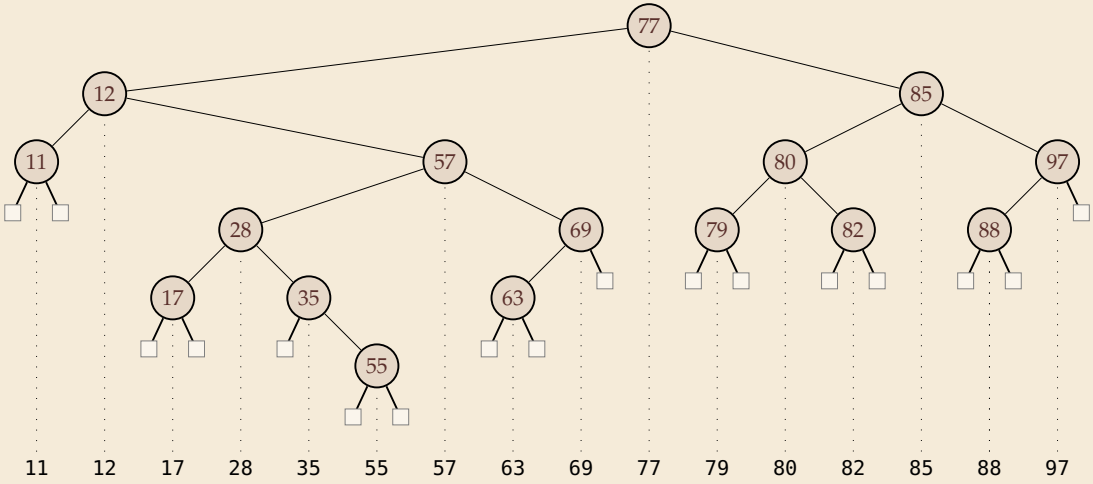
# BST insert

**Example:** Insert 88



# BST delete

- Easy case: remove leaf, e. g., 11 ⇝ replace by `null`
- Medium case: remove unary, e. g., 69 ⇝ replace by unique child
- Hard case: remove binary, e. g., 85 ⇝ swap with successor, recurse



# Analysis

Let $h$ be the *height* of the BST (longest root-to-leaf path, in edges).

- **Search:** follows one root-to-node path, one comparison per node

$\rightsquigarrow$ $\leq h + 1$ comparisons $\rightsquigarrow$ $O(h)$

- **Insert:** unsuccessful search for key ($\leq h + 1$ cmps), then attach new leaf $\rightsquigarrow$ $O(h)$
- **Delete:**
  - search for the key ($O(h)$)
  - leaf / unary case: $O(1)$ pointer changes
  - binary case: find predecessor (walk down the left subtree, $O(h)$), swap, delete the predecessor (leaf or unary) $\rightsquigarrow$ $O(h)$

$\rightsquigarrow$ everything is $O(h)$ . . . *so how large is* $h$*?*

# What is the height of a BST?

**Worst Case:**

- $h = n - 1 = \Theta(n)$

**Average Case:**

- Assumption: insertions come in random order
  no deletions

$\rightsquigarrow$

# Random BSTs – Average Search Cost

**Model:** $n$ distinct keys inserted in *random order* (all $n!$ orders equally likely), no deletions.

Let $C_n$ := expected total cost of searching *each* key once (cost = #nodes on search path).

- root = first inserted key $\rightsquigarrow$ its rank $r$ is uniform in $[1..n]$
- left/right subtree: random BSTs of $r-1$ resp. $n-r$ keys
  (remaining keys still arrive in random order)

$$\rightsquigarrow\quad C_n \;=\; n \;+\; \frac{1}{n}\sum_{r=1}^{n}\bigl(C_{r-1}+C_{n-r}\bigr) \qquad (n \geq 1), \qquad C_0 = 0$$

comparison with root, once per key

# Random BSTs – Getting Rid of the Full History

$$C_n \;=\; n \;+\; \frac{1}{n}\sum_{r=1}^{n}\bigl(C_{r-1}+C_{n-r}\bigr) \;=\; n+\frac{2}{n}\sum_{k=0}^{n-1}C_k \qquad (n\geq 1), \qquad C_0=0$$

$$nC_n \;=\; n^2+2\sum_{k=0}^{n-1}C_k$$

$$nC_n \;-\; (n-1)C_{n-1} \;=\; n^2-(n-1)^2 \;+\; \overbrace{2\sum_{k=0}^{n-1}C_k \;-\; 2\sum_{k=0}^{n-2}C_k}^{=2C_{n-1}}$$

$$\Longleftrightarrow \qquad nC_n \;=\; 2n-1+(n+1)C_{n-1} \qquad \Big|\,:n(n+1)$$

$$\frac{C_n}{n+1} \;=\; \frac{2n-1}{n(n+1)} \;+\; \frac{C_{n-1}}{n} \qquad (n\geq 1)$$

Abbreviate $\boxed{D_n := \dfrac{C_n}{n+1}}$ $\qquad D_n = \dfrac{2n-1}{n(n+1)} + D_{n-1} \quad (n\geq 1), \qquad D_0=0$

# Random BSTs – Telescoping Recurrence

$$D_n = \sum_{k=1}^{n} \frac{2k-1}{k(k+1)} \qquad \frac{2k-1}{k(k+1)} = \frac{-1}{k} + \frac{3}{k+1} \quad \text{(partial fractions)}$$

check: $\frac{-(k+1)+3k}{k(k+1)} = \frac{2k-1}{k(k+1)}$ ✓

$$D_n = \underbrace{-\sum_{k=1}^{n} \frac{1}{k}}_{=H_n} + 3 \underbrace{\sum_{k=1}^{n} \frac{1}{k+1}}_{=H_{n+1}-1} = 3H_{n+1} - 3 - H_n = 2H_n - 3 + \frac{3}{n+1}$$

(using $H_{n+1} = H_n + \frac{1}{n+1}$)

$$C_n = (n+1)D_n = 2(n+1)H_n - 3(n+1) + 3 = 2(n+1)H_n - 3n$$

$$\boxed{H_n = \ln n + \gamma + O\left(\tfrac{1}{n}\right)}$$ (harmonic numbers; $\gamma \approx 0.5772$)

$\rightsquigarrow$ $C_n \sim 2n \ln n \approx 1.39\, n \lg n$ ($\ln n = \lg n / \lg e$, $2/\lg e \approx 1.386$)

$\rightsquigarrow$ average cost per search $\frac{C_n}{n} \sim 2 \ln n \approx 1.39 \lg n = \Theta(\log n)$

$\neq$ *height*: $\mathbb{E}[h] \sim 4.311 \ln n$; also $\Theta(\log n)$, but much harder proof.

**Devroye**: *A note on the height of binary search trees*, J. ACM 33(3) **1986**

# BST summary

| Operation | Running Time | Worst Case ($h = n - 1$) | Average Case (random insertion order) |
|---|---|---|---|
| `construct(`$A[1..n]$`)` | $O(nh)$ | $O(n^2)$ | $O(n \log n)$ |
| `put(`$k$`,`$v$`)` | $O(h)$ | $O(n)$ | $O(\log n)$ |
| `get(`$k$`)` | $O(h)$ | $O(n)$ | $O(\log n)$ |
| `delete(`$k$`)` | $O(h)$ | $O(n)$ | $O(\log n)$ |
| `contains(`$k$`)` | $O(h)$ | $O(n)$ | $O(\log n)$ |
| `isEmpty()` | $O(1)$ | $O(1)$ | $O(1)$ |
| `size()` | $O(1)$ | $O(1)$ | $O(1)$ |

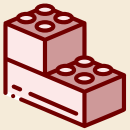
**Average case:**

- $n$ distinct keys inserted in random order, no deletions;
  then $\mathbb{E}[h] \sim 4.311 \ln n$ and the average search path has $\sim 2 \ln n \approx 1.39 \lg n$ nodes (see above).
- $h = \Theta(\log n)$ "with high probability":
  $\forall d \; \exists c : \Pr[h \geq c \lg(n)] \leq n^{-d}$
- With (Hibbard) deletions interleaved, the expected height degrades to $\Theta(\sqrt{n})$ – balanced BSTs (§3.8) fix both columns.

# 3.7 Ordered Symbol Tables

# Ordered symbol tables

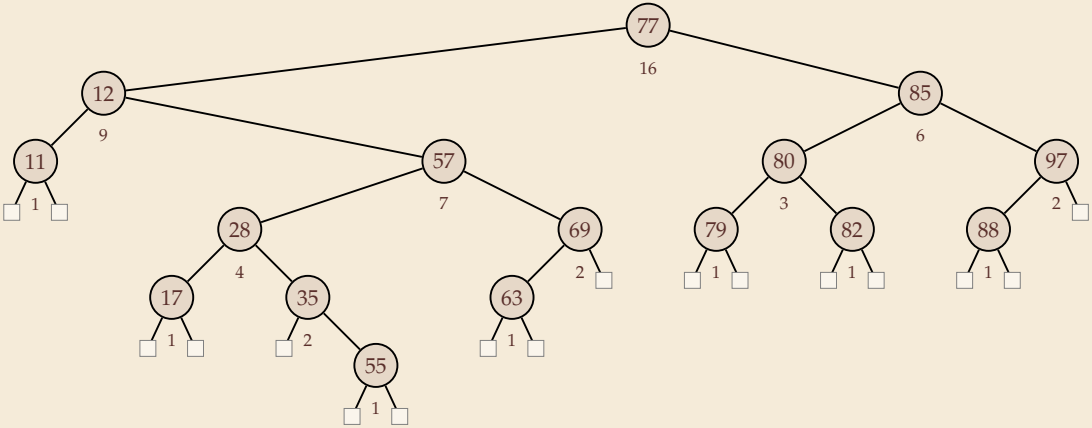
- `min()`, `max()`
  Return the smallest resp. largest key in the ST
- `floor(`$x$`)`, $\lfloor x \rfloor = \mathbb{Z}$.`floor(`$x$`)`
  Return largest key $k$ in ST with $k \leq x$.
- `ceiling(`$x$`)`
  Return smallest key $k$ in ST with $k \geq x$.
- `rank(`$x$`)`
  Return the number of keys $k$ in ST $k < x$.
- `select(`$i$`)`
  Return the $i$th smallest key in ST (zero-based, i. e., $i \in [0..n)$)



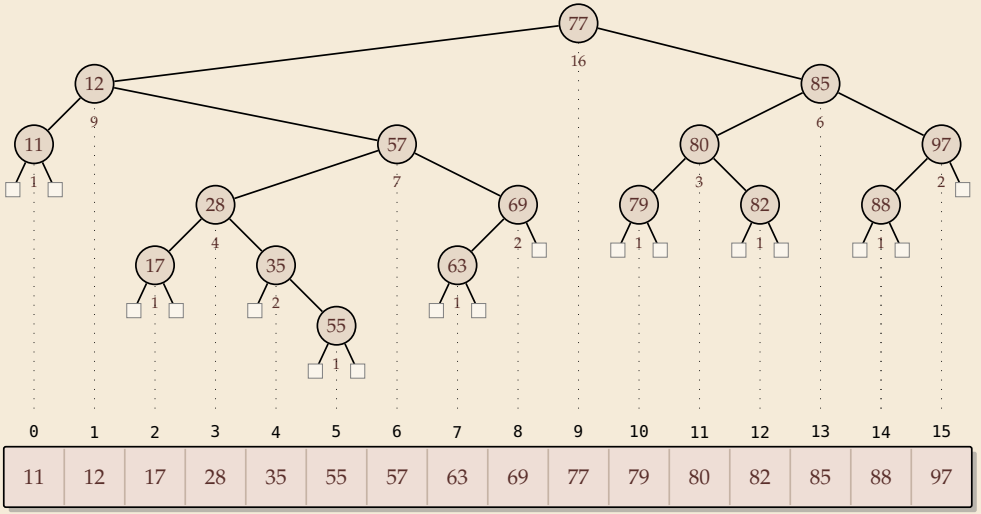
*With select, we can simulate access as in a truly dynamic array*!

(Might not need any keys at all then!)

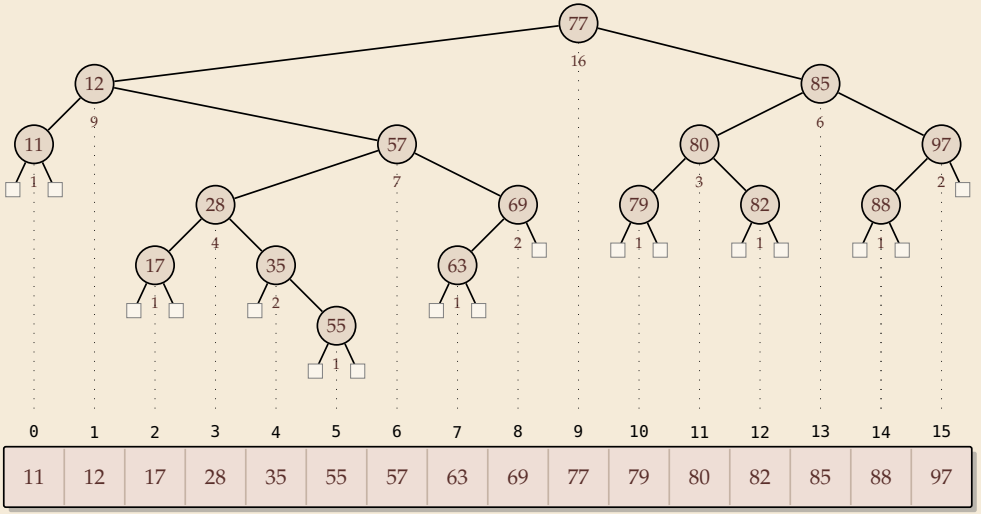
# Augmented BSTs



# Rank


77
16
12
9
85
6
11
1
57
7
80
3
97
2
28
4
69
2
79
1
82
1
88
1
17
1
35
2
63
1
55
1
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
11 12 17 28 35 55 57 63 69 77 79 80 82 85 88 97

# Select


77
16
12
9
85
6
11
1
57
7
80
3
97
2
28
4
69
2
79
1
82
1
88
1
17
1
35
2
63
1
55
1
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
11 12 17 28 35 55 57 63 69 77 79 80 82 85 88 97

# Why store subtree sizes?

- Note that in an augmented BST, each node stores the **size of its subtree**.
- . . . why not directly store the **rank**? Would make rank/select much simpler!

- Problem: Single insertion/deletion can change *all* node ranks!

$\rightsquigarrow$ Cannot efficiently maintain node ranks.

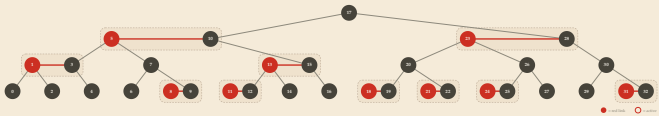

Subtree sizes only change along search path $\rightsquigarrow$ $O(h)$ nodes affected

# 3.8 Balanced BSTs

# Balanced BSTs

**Balanced binary search trees**:

- imposes shape invariant that guarantees $O(\log n)$ height
- adds rules to restore invariant after updates
- many examples known
  - *AVL trees* (height-balanced trees)
  - *red-black trees*
  - *weight-balanced trees* (BB[$\alpha$] trees)
  - ...



**Other options:**

I'd love to talk more about all of these ...
(Maybe another time)

- **amortization:** *splay trees*, *scapegoat trees*
  *COLA (cache oblivious lookahead array)*
- **randomization:** *randomized BSTs*, *treaps*, *skip lists*

# BSTs vs. Heaps

**Balanced binary search tree**

| Operation | Running Time |
|---|---|
| `construct(`$A[1..n]$`)` | $O(n \log n)$ |
| `put(`$k$`,`$v$`)` | $O(\log n)$ |
| `get(`$k$`)` | $O(\log n)$ |
| `delete(`$k$`)` | $O(\log n)$ |
| `contains(`$k$`)` | $O(\log n)$ |
| `isEmpty()` | $O(1)$ |
| `size()` | $O(1)$ |
| `min()` / `max()` | $O(\log n) \rightsquigarrow O(1)$ |
| `floor(`$x$`)` | $O(\log n)$ |
| `ceiling(`$x$`)` | $O(\log n)$ |
| `rank(`$x$`)` | $O(\log n)$ |
| `select(`$i$`)` | $O(\log n)$ |

~~**Binary heaps**~~ *Strict Fibonacci heaps*

| Operation | Running Time |
|---|---|
| `construct(`$A[1..n]$`)` | $O(n)$ |
| `insert(`$x$`,`$p$`)` | ~~$O(\log n)$~~ $O(1)$ |
| `delMax()` | $O(\log n)$ |
| `changeKey(`$x$`,`$p'$`)` | ~~$O(\log n)$~~ $O(1)$ |
| `max()` | $O(1)$ |
| `isEmpty()` | $O(1)$ |
| `size()` | $O(1)$ |

- apart from faster `construct`, BSTs always as good as binary heaps
- MaxPQ abstraction still helpful
- and faster heaps exist!

# 3.9 Hashing – Basics

# Lower bound for search

The fastest implementations of the ordered symbol table ADT require $\Theta(\log n)$ time to search among $n$ items. *Is this the best possible?*

- **Theorem:** In the comparison model (on the keys), $\Omega(\log n)$ comparisons are required to search a size-$n$ dictionary.

- *Proof:* Any algorithm defines a *decision tree*: internal nodes are comparisons, leaves are outcomes.
  There are $n + 1$ different outcomes ("is $x_i$" or "not found"), so $\geq n + 1$ leaves;
  a binary tree with $\ell$ leaves has height $\geq \lceil \lg \ell \rceil$ (ADS). ■

$\rightsquigarrow$ Balanced BSTs are optimal . . . *in the comparison model.*

$\rightsquigarrow$ To beat $\Theta(\log n)$ we must leave that model:
*compute* with the key instead of only comparing it.

(and we will only get *expected* $O(1)$ – the lower bound is for the worst case)

# Hashing Recap

**Notation:** $U = [0..u)$ universe of keys; $x_1, \ldots, x_n$ the $n$ stored keys;
$T[0..m)$ the table; $\alpha = n/m$ the *load factor*;
$h : U \to [0..m)$

- store $x$ (and its value) at or near $T[h(x)]$

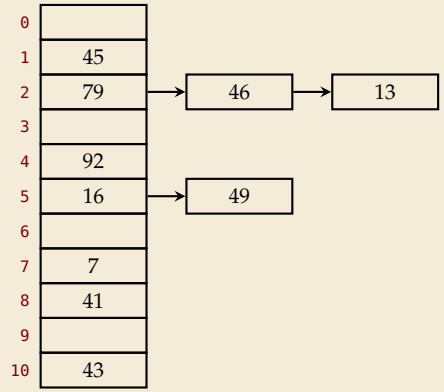
⇝ `get/put/delete` become array accesses . . .

↯ . . . if no two stored keys share an index.

$u \gg m$ ⇝ collisions *unavoidable* (pigeonhole),
and *likely*: already $n \approx 1.18\sqrt{m}$ random keys
collide with probability $\geq \frac{1}{2}$ (birthday paradox)

- Collision handling (details: ADS / S&W §3.4)
  - *separate chaining*: $T[i]$ holds list keys with $h(x) = i$
  - *open addressing*: probe $T[h(x)], T[h(x)+1], \ldots$
    linear, or other probe sequences
    (quadratic, Robin Hood, cuckoo, . . . )

$m = 11$, $h(x) = x \bmod 11$; chaining



# Hashing Recap [2]

**Notation:** $U = [0..u)$ universe of keys; $x_1, \ldots, x_n$ the $n$ stored keys;
$T[0..m)$ the table; $\alpha = n/m$ the *load factor*;
$h : U \to [0..m)$

- *rehash* into a new table when $\alpha$ leaves $[\alpha_{\min}, \alpha_{\max}]$
  $\rightsquigarrow$ $\Theta(n + m)$, amortized $O(1)$ (doubling trick!)

- **Cost** of an operation $\approx$ #keys examined $=$ length of the chain / probe sequence.

  *How long are they?*

  - in worst case: all keys(!)

# Uniform hashing assumption

- ***Simple Uniform Hashing Assumption:***
  For the keys $x_1, \ldots, x_n$ in our dictionary we have

$$\mathop{\mathbb{P}}_{x_i \neq x_j} \left[ h(x_i) = h(x_j) \right] = \frac{1}{m}$$

$\approx$ keys in the input are drawn i.i.d. uniformly from $U = [0..u)$,
and $h$ spreads $U$ evenly over $[0..m)$.

- This is an assumption about the *input*. We will come back to that.

# Analysis of chaining (average case)

- $L_i$: # keys in bucket $i$
  - how many keys hash to $i$?
- unsuccessful search: $1 + L_{h(x)}$ memory accesses
- successful search: $\leq 1 + L_{h(x_i)}$ memory accesses

$\rightsquigarrow$ key quantity: $\mathbb{E}[L_{h(x_i)}]$

- Define $X_{i,j} = \mathbb{1}\{h(x_i) = h(x_j)\}$ $\rightsquigarrow$ $L_{h(x_i)} = \sum_{j=1}^{n} X_{i,j}$
  - indicator random variable "keys $x_i$ and $x_j$ collide"
  - deterministically $X_{i,i} = 1$
  - *uniform hashing* implies

    for $i \neq j$: $\mathbb{P}\big[h(x_i) = h(x_j)\big] = \frac{1}{m}$ $(*)$

expectation over the *input*

$$
\begin{aligned}
\mathbb{E}[L_{h(x_i)}] &= \mathbb{E}\Big[\sum_{j=1}^{n} X_{i,j}\Big] \\
&\underset{\text{lin. } \mathbb{E}}{=} \sum_{j=1}^{n} \mathbb{E}[X_{i,j}] \\
&= \sum_{j=1}^{n} \mathbb{P}[X_{i,j} = 1] \\
&= 1 + \sum_{j \neq i} \mathbb{P}[h(x_i) = h(x_j)] \\
&\underset{(*)}{=} 1 + \sum_{j \neq i} \frac{1}{m} \\
&\leq 1 + \alpha
\end{aligned}
$$

# 3.10 Universal Hashing

# Discussion

- Running time for hashing with chaining is $O(1)$ if we keep $m$ proportional to $n$

$\rightsquigarrow$ see amortized analysis of the *doubling trick*

- `java.util.HashMap` works like this!

  + red-black tree for large buckets

- **But:** only in expectation, and only under the **assumption** of *uniform hashing*!

  - Worst case: all keys collide in one bucket.
  - For *every fixed* $h$ this worst case exists: $u \geq n \cdot m$ $\rightsquigarrow$ some bucket contains $\geq n$ keys of $U$ (pigeonhole)
  - An adversary who knows $h$ inserts exactly those $\rightsquigarrow$ every operation $\Theta(n)$.
    (Not hypothetical: hash-flooding attacks on Perl, PHP, Java `hashCode`, ... – cf. Sedgewick & Wayne)
  - *Can we avoid this?*



*Randomization!*



# Universal families of hash functions

- **Idea:** Choose $h$ **at random** from a set $\mathcal{H}$.
- A set $\mathcal{H} \subseteq \{h : U \to [0..m)\}$ of hash functions is ***c-universal*** if for a (uniformly) random $h \in \mathcal{H}$ we have

$$\forall x \neq x' \; : \; \mathop{\mathbb{P}}_{h \in \mathcal{H}}\big[h(x) = h(x')\big] \; \leq \; \frac{c}{m}$$

- universal = 1-universal
- Compare 1-universal with

***Simple Uniform Hashing Assumption:***
For the keys $x_1, \ldots, x_n$ in our dictionary we have

$$\mathop{\mathbb{P}}_{x_i \neq x_j}\big[h(x_i) = h(x_j)\big] \; = \; \frac{1}{m}$$

$\rightsquigarrow$ essentially the same collision probability!

**But:** instead of *hoping* for a benign input (average case), we *enforce* the average case by randomization – for *every* input.

# Analysis of chaining (randomized)

- Choose $h$ *at random* from a $c$-universal $\mathcal{H}$. *Then* let the adversary pick the keys $x_1, \ldots, x_n$ and $x$.
- $L_i$: # keys in bucket $i$
  - unsuccessful search: $1 + L_{h(x)}$ memory accesses
  - successful search: $\leq 1 + L_{h(x_i)}$ memory accesses
  - $\rightsquigarrow$ key quantity: $\mathbb{E}[L_{h(x_i)}]$
- Define $X_{i,j} = \mathbb{1}\{h(x_i) = h(x_j)\} \quad \rightsquigarrow \quad L_{h(x_i)} = \sum_{j=1}^{n} X_{i,j}$
  - indicator random variable "keys $x_i$ and $x_j$ collide"
  - deterministically $X_{i,i} = 1$
  - *universality* implies

    for $x \neq x'$: $\mathbb{P}_{h \in \mathcal{H}}\big[h(x) = h(x')\big] \leq \frac{c}{m}$ $\quad (*)$

expectation over our *random bits*

$$
\begin{aligned}
\mathbb{E}[L_{h(x_i)}] &= \mathbb{E}\Big[\sum_{j=1}^{n} X_{i,j}\Big] \\
&\underset{\text{lin. } \mathbb{E}}{=} \sum_{j=1}^{n} \mathbb{E}[X_{i,j}] \\
&= \sum_{j=1}^{n} \mathbb{P}[X_{i,j} = 1] \\
&= 1 + \sum_{j \neq i} \mathbb{P}[h(x_i) = h(x_j)] \\
&\underset{(*)}{\leq} 1 + \sum_{j \neq i} \frac{c}{m} \\
&\leq 1 + c\alpha
\end{aligned}
$$

Same analysis – only *what is random* has changed.

Adversary may know $\mathcal{H}$, but not which $h$ we drew; re-draw $h$ at every rehash.

# Do universal families exist?

- Yes! e. g., the set of **all** functions $\mathcal{H} = \{h : U \to [0..m)\}$
  - With $h \in \mathcal{H}$ uniform, hash values behave exactly like i. i. d. uniform random variables: $\forall x \in U : h(x) \overset{\mathcal{D}}{=} \text{Uniform}[0..m)$

  $\rightsquigarrow \underset{h \in \mathcal{H}}{\mathbb{P}}\left[h(x) = h(x')\right] = \frac{1}{m}$

  $\rightsquigarrow$ $\mathcal{H}$ is 1-universal

↯ too expensive! We need $\Theta(u)$ space to store $h$ ...

- Wanted: a *small* family $\rightsquigarrow$ $h$ fits in $O(1)$ words and is $O(1)$ to evaluate

# Dot-product hashing

- **Assumption:** $m$ is prime
- **Assumption:** $u = m^r$ for some $r \in \mathbb{N}_{\geq 1}$

$\rightsquigarrow$ key $x$ corresponds to a number with $r$ digits in base $m$: $x = \langle x_{r-1}, x_{r-2}, \ldots, x_0 \rangle$

- For $a \in [0..u)$, define $h_a(x) = (a^T x) \bmod m = \sum_{l=0}^{r-1} a_l x_l \bmod m$

  dot product of $a$ and $x$ as $r$-dimensional vectors

- $\mathcal{H} = \{h_a \mid a \in [0..u)\}$

- **Properties:**
  - $h_a$ can be stored in $O(\log u)$ bits (store $a$!)
  - $h_a$ is computed in $O(1)$ time in the word-RAM with word size $w = \Omega(\log u)$

    $\rightsquigarrow$ keys fit in $O(1)$ words

# Dot-product hashing is universal

**Theorem 3.1**

Let $m$ be prime and $u = m^r$ for $r \in \mathbb{N}_{\geq 1}$. Then $\mathcal{H} = \{h_a \mid a \in [0..u)\}$ is 1-universal. ◀

**Proof:**

$x \neq x'$ arbitrary but fixed.

To show: $\mathbb{P}_{a \in [0..m)^r}\left[h_a(x) = h_a(x')\right] \leq \frac{1}{m}$

$x = \langle x_{r-1}, \ldots, x_0 \rangle \neq \langle x'_{r-1}, \ldots, x'_0 \rangle = x'$
$\rightsquigarrow \exists d : x_d \neq x'_d.$

$m$ prime $\rightsquigarrow$ $\mathbb{Z}_m$ is a field
$\rightsquigarrow$ multiplicative inverses:
$\forall y \neq 0\ \exists y^{-1} : y \cdot y^{-1} \equiv 1 \pmod{m}$ (†)

Shorthand: $a_{\neq d} = \langle a_{r-1}, \ldots, a_{d+1}, a_{d-1}, \ldots, a_0 \rangle$

$$
\begin{aligned}
&\mathbb{P}_{a \in [0..m)^r}\left[h_a(x) = h_a(x')\right] \\
&= \mathbb{P}_{a \in [0..m)^r}\left[\sum_{i=0}^{r-1} a_i x_i \equiv \sum_{i=0}^{r-1} a_i x'_i \pmod{m}\right] \\
&= \mathbb{P}_{a \in [0..m)^r}\left[a_d(x_d - x'_d) \equiv \sum_{i \neq d} a_i (x'_i - x_i) \pmod{m}\right] \\
&\underset{(\dagger)}{=} \mathbb{P}_{a \in [0..m)^r}\Big[a_d \equiv \underbrace{(x_d - x'_d)^{-1} \sum_{i \neq d} a_i (x'_i - x_i)}_{C(x, x', a_{\neq d})} \pmod{m}\Big] \\
&\underset{\text{law of total prob.}}{=} \mathbb{E}_{a_{\neq d}}\left[\mathbb{P}_{a_d \in [0..m)}\left[a_d \equiv C(x, x', a_{\neq d}) \pmod{m} \,\middle|\, a_{\neq d}\right]\right] \\
&= \mathbb{E}_{a_{\neq d}}\left[\frac{1}{m}\right] = \frac{1}{m} \quad \blacksquare
\end{aligned}
$$

# Linear congruential hashing

## Theorem 3.2 (Linear congruential hashing)

Let $p \geq u$ be prime and $m \leq p$. Then

$$\mathcal{H}_{p,m} \;=\; \left\{ h_{a,b} \;\middle|\; a \in [1..p),\ b \in [0..p),\ \ h_{a,b}(x) = \big((ax+b) \bmod p\big) \bmod m \right\}$$

is 1-universal.

**Proof:**

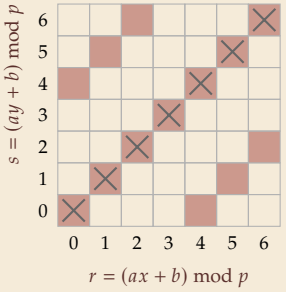
Let $x \neq y$ in $U$ be arbitrary but fixed.

Write $r = (ax+b) \bmod p,\ \ s = (ay+b) \bmod p$.

- We have $r \neq s$: $r - s \equiv a(x-y) \not\equiv 0 \pmod p$, since $a \neq 0$, $x \not\equiv y$ and $p \geq u$ prime.
- Further, $(a,b) \mapsto (r,s)$ is a *bijection* onto the $p(p-1)$ pairs $r \neq s$: $a = (r-s)(x-y)^{-1} \bmod p,\ \ b = (r - ax) \bmod p$.

$\rightsquigarrow$ uniform $(a,b)$ = uniform *off-diagonal* cell $(r,s)$.

all pairs $(r,s)$ for $p = 7$, $m = 4$




shaded = collision ($r \equiv s$);
× diagonal = impossible ($r \neq s$).

# Proof [2]

**Proof (cont.):**

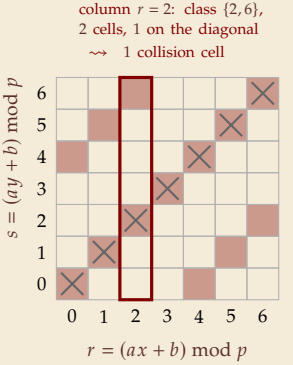
- collision iff $r \equiv s \pmod m$: the shaded cells.

  Per column $r$: a residue class of $\le \lceil p/m \rceil$ cells, *one of which is the diagonal*

  $\rightsquigarrow \quad \le \lceil p/m \rceil - 1 \le \dfrac{p-1}{m}$ shaded off-diagonal cells.

  shaded off-diagonal: $2+2+2+0 = 6$ of 42 cells $\rightsquigarrow \ \frac{1}{7} \le \frac{1}{4}$ ✓

$\rightsquigarrow \ \mathbb{P}[h_{a,b}(x) = h_{a,b}(y)] \ \le \ \dfrac{p \cdot \frac{p-1}{m}}{p(p-1)} \ = \ \dfrac{1}{m}.$




all pairs $(r, s)$ for $p = 7$, $m = 4$

**Note:** *Crude count* (all shaded cells, diagonal included, $a = 0$ allowed): 13 of $49 \approx 0.27 > \frac{1}{4}$ $\rightsquigarrow$ only $c$-universal with $c = \left(\lceil \frac{p}{m} \rceil / \frac{p}{m}\right)^2 \le 4$. The bijection is what lets us drop the diagonal.

# Other universal families

## Theorem 3.3 (Multiplicative hashing)

For $w$-bit keys and $m = 2^d$, the family
$\left\{ h_z(x) = \left((z \cdot x) \bmod 2^w\right) \operatorname{div} 2^{w-d} \;\middle|\; z \text{ odd},\ z \in [1..2^w) \right\}$ is 2-universal. ◀



**Dietzfelbinger, Hagerup, Katajainen, Penttonen**: *A Reliable Randomized Algorithm for the Closest-Pair Problem*, J. Algorithms **1997**

*In practice:* Python randomizes its string hash per process (SipHash, since 3.3) for exactly this reason; Java does not, and instead turns long chains into red-black trees.

# Hashing with chaining – Summary

**Average case → expectation in the worst case**

- With a *universal hash family* we achieve the same complexity as under the uniform hashing assumption –

  **but:** without any assumption on the input!

- analogous to Quicksort with random pivot choice (Unit 4)

- total running time is still amortized (doubling trick for rehashing!)


fastest (amortized) access time


simple data structure


somewhat wasteful with space


worst case $O(n)$ per operation (cuckoo hashing fixes this for searches)

# Comparison of Classic Hashing Schemes

| Hash table design | Search hit | Search miss | Insert | Space | good $\alpha = \frac{n}{m}$ |
|---|---|---|---|---|---|
| Separate Chaining | $\approx 1 + \frac{1}{2}\alpha$ | $\approx \alpha$ | = miss | $n + m$ | $\approx 2$ |
| Linear Probing | $\approx \frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right)$ | $\approx \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$ | = miss | $m$ | $\leq 0.5$ |
| Quadratic Probing | $\approx 1 + \ln\left(\frac{1}{1-\alpha}\right) - \frac{1}{2}\alpha$ | $\approx \frac{1}{1-\alpha} - \alpha + \ln\left(\frac{1}{1-\alpha}\right)$ | = miss | $m$ | $\leq 0.7$ |
| Robin Hood (Lin. P.) | = Lin. P.; Var. $O(1)$ | $\leq$ Lin. P. | = miss | $m$ | $\leq 0.9$ |
| Robin Hood Clever | $O(1)$ | $O(1)$ | = miss | $m$ | $\leq 1$ (=any!) |
| $d$-way Cuckoo H. | $\leq d$ worst case | $\leq d$ worst case | amort. | $m$ | $< c_d$ |

$c_2 = \frac{1}{2}$, $c_3 \approx 0.918$

- Assumption: uniform hashing (all $m^n$ hash sequences equally likely)
- Cost: expected # (equality) comparisons, for $n, m \to \infty$ with $\alpha$ fixed
- Space usage in words on top of space for items (without space for optional optimizations)
- Robin Hood only *reorders* within clusters: same mean as linear probing, but the cost of a search is *concentrated* (and with random probing + "mean-centred" search even $O(1)$ expected in a full table [Poblete & Viola 2019])

More improvements possible with word-RAM bitwise tricks $\rightsquigarrow$ *Advanced Data Structures*

# Hashing vs. Balanced Search Trees

**Advantages of Balanced Search Trees**

- $O(\log n)$ worst-case operation cost
- Does not require any assumptions, special functions, or known properties of input distribution
- Predictable (and often smaller) space usage (exactly $n$ nodes)
- Never need to rebuild the entire structure
- supports ordered dictionary operations (rank, select etc.)

**Advantages of Hash Tables**

- $O(1)$ operations (if hashes well-spread and load factor small)
- We can choose space-time tradeoff via load factor
- Cuckoo hashing achieves $O(1)$ worst-case for search & delete